

中山大學

SUN YAT-SEN UNIVERSITY

项目报告

Web-VLMAgent：多模态网页智能体

姓 名　王一澄、王胜伟、吴乐超

学 号　23336233、23336228、23336250

学 院　计算机学院

专 业　计算机科学与技术

2026 年 6 月 22 日

摘　要

本项目实现面向真实网页任务的多模态 Agent 系统 Web-VLMAgent。系统以自然语言任务和起始网页为输入，通过 Playwright 控制浏览器，组合原始截图、Set-of-Marks 标注图、DOM 候选元素、Accessibility Tree、页面文本及动作历史；模型采用 ReAct 风格单步决策，输出结构化 JSON 动作。执行侧完成动作校验、安全检查、进度监测及最终验证，并保存逐步 trace，支持 YAML 任务管理、批量运行、结果校验和 HTML 回放。

任务体系分为 **8 项核心评测任务与 1 项补充探索任务**。核心任务覆盖 CSRankings 信息检索、Bilibili 内容分析和广州旅游规划三个场景；CSRankings 跨方向排名与 faculty 统计单独作为探索性任务，其题目与判分设置仍需审查，不纳入核心完成率。DeepSeek V4 Pro 在每项核心任务上重复运行 10 次，共 80 次，记录到 76 次 harness 完成，完成率为 **95.00%**。GPT-5.5 的视觉基线在 8 项核心任务上各运行一次，均完成，平均 9.75 步；其中两项 CSRankings 核心任务通过确定性校验，六项开放任务通过字段覆盖检查，仍需结合 trace 复核内容真实性。

项目另开展十种模型配置的横向实验，覆盖视觉与纯文本观察。原始批次包含核心和探索两部分，共 90 个计划组合；历史汇总记录为 66 个有效、22 个无效和 2 个未完成。由于历史汇总未完整区分每个模型的探索任务状态，这些数值仅用于说明原始批次覆盖，不作为八任务核心评测的分母或排行榜。

本报告给出系统架构、观察与动作空间、核心评测口径、探索任务范围及失败类型分析。完成率反映 Agent 主流程是否结束并生成答案；确定性任务采用答案规则校验，开放任务则需要字段、来源及轨迹证据共同复核。实验说明真实网页 Agent 的评估应同时报告任务范围、完成结果、校验方式与环境异常，避免将流程完成等同于答案正确。

关键词：网页 Agent；多模态大模型；Playwright；Set-of-Marks；ReAct；批量评测

目录

1 引言 **1**
1.1 任务背景 . . . 1
1.2 问题定义 . . . 2
1.3 项目贡献 . . . 2

2 相关工作 **3**
2.1 真实网页任务与评测环境 . . . 3
2.2 界面理解与 Grounding . . . 3
2.3 推理-行动闭环 . . . 3
2.4 GUI Agent 的扩展启发 . . . 4
2.5 本项目与已有工作的区别 . . . 4

3 系统设计与实现 **5**
3.1 总体架构 . . . 5
3.2 观察空间 . . . 5
3.3 候选元素与 Set-of-Marks . . . 6
3.4 动作空间与安全校验 . . . 6
3.5 ReAct Prompt 与单步决策 . . . 7
3.6 Progress Checker 与 Final Verifier . . . 8
3.7 Trace 格式与可视化 . . . 8

4 任务设计与评测方案 **9**
4.1 任务库设计原则 . . . 9
4.2 难度划分 . . . 10
4.3 评测指标与判分口径 . . . 11
4.4 第一轮评测：DeepSeek V4 Pro 八项核心任务重复评测 . . . 12
4.5 GPT-5.5 八项核心任务视觉基线 . . . 13
4.6 十模型横向评测 . . . 14
4.7 十模型评测中的无效样本与风险 . . . 15

5 错误分析、局限与展望 **16**
5.1 从有效样本看失败模式 . . . 16
5.2 失败类型与改进方向 . . . 16
5.3 当前局限 . . . 17
5.4 后续工作 . . . 17
5.5 总结 . . . 17

1 引言

1.1 任务背景

多模态大模型正在从“看图回答问题”走向“理解界面并执行操作”。网页是最典型也最复杂的图形用户界面之一：页面同时包含文本、按钮、表单、弹窗、图片、布局层级和动态加载内容；用户任务又往往不是单步问答，而是需要跨页面搜索、筛选、比较、记录证据并生成最终答案。因此，网页 Agent 的难点不只是让模型读懂页面，而是让模型在真实浏览器环境中稳定完成“观察、规划、操作、检查、验证”的闭环。

本课程题目 7 要求设计网页多模态 Agent：基于网页截图/结构信息和自然语言指令，生成点击、输入、滚动等浏览器操作并完成具体任务。课程给出的最低验收标准包括：选择 1–2 个真实网站，每个网站至少 3 个任务场景；每个场景至少 10 次独立测试，总测试不少于 60 次；报告任务成功率；分类记录识别失败、规划失败和执行失败；提供截图序列与动作序列等可视化操作轨迹。

我们实现的 Web-VLMAgent 将该要求进一步工程化：系统不只演示若干成功案例，而是把网页任务抽象为可配置的 YAML episode，把每一步观察、模型响应、结构化动作、执行结果、进展检查和完成验证都写入 trace，从而支持批量评测、错误分析和轨迹回放。

表 1 课程最低要求与本项目实现目标对照

维度	课程最低要求	本项目实现与扩展
网站与任务	1–2 个真实网站，每站至少 3 个任务	3 类真实网站/场景，8 项核心评测任务与 1 项补充探索任务，按难度分层
运行方式	浏览器自动化演示	Playwright/CDP 浏览器环境，结构化动作执行，支持 headless 与可视化运行
观察信息	网页截图/结构信息	原始截图、SoM 标注截图、DOM 候选元素、Accessibility Tree、页面文本、动作历史
评测指标	至少成功率	成功率、平均步数、平均用时、动作 JSON 合法率、定位错误率、纠错成功率、失败类型分布
轨迹材料	截图序列 + 动作序列	每步保存 screenshot、marked screenshot、candidates、prompt、raw response、parsed action、execution result、progress check、final verification
安全约束	未明确展开	拦截登录、支付、购买、删除、敏感信息提交、验证码绕过等高风险行为

1.2 问题定义

一个网页任务 episode 由五部分组成：

1. 起始地址 u_0，例如 CSRankings 的过滤页面或 B 站搜索结果页；
2. 自然语言任务指令 I，描述用户希望获得的最终结果；
3. 最大步数 T，防止模型陷入无限循环；
4. 成功标准 C，可以是确定性答案、字段覆盖、页面证据数量或开放式人工复核标准；
5. 禁止动作集合 F，例如登录、支付、下单、删除、提交敏感信息等。

在第 t 步，Agent 从浏览器获得观察 o_t，模型输出结构化动作 a_t，执行器验证并执行动作，浏览器转移到新状态 s_{t+1}。若模型输出 `answer` 或 `done`，系统调用 Final Verifier 检查最终答案是否有页面证据支持；若达到最大步数或被判定为卡住，系统生成失败/部分完成结果并保留 trace。整个过程可以概括为：

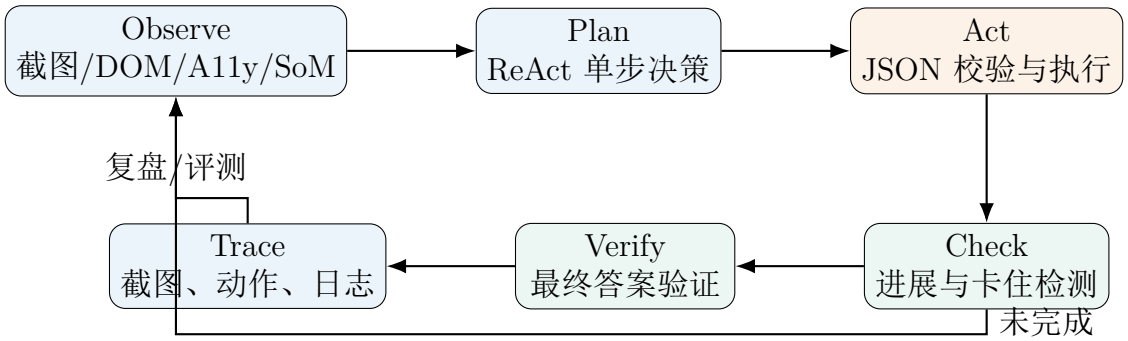


图 1 Web-VLMAgent 的闭环问题定义

1.3 项目贡献

本项目的主要贡献如下。

1. **实现了最小可运行到批量评测的完整闭环。**从 Phase 1 的 Playwright 浏览器环境、候选元素提取和 mock agent，到 Phase 5 的 YAML 任务管理、批量 runner、指标统计与 HTML trace report，形成了端到端可运行系统。
2. **设计了混合观察空间。**对视觉模型提供原始截图和 SoM 标注图，对纯语言模型提供候选元素、页面文本和 Accessibility Tree，使不同能力模型可以在同一动作接口下比较。
3. **将动作空间约束为结构化 JSON。**模型不能执行任意代码，也不能直接发出鼠标坐标；所有 click/type 都必须引用当前候选元素的 `target_id`，并通过安全校验。
4. **构造了 8 项核心任务与 1 项补充探索任务。**核心评测覆盖 CSRankings 信息检索、Bilibili 内容分析与广州旅游规划三个场景；另保留 CSRankings 跨方向统计作为探索性任务，用于检查任务定义和长链证据组织，不纳入核心完成率。

5. **评测可追溯可复盘。**每一次运行都保存截图序列、标注图、候选元素、prompt、模型原始输出、解析后动作和执行结果；评测报告可以定位失败发生在识别、规划、执行、验证还是网站环境环节。

2 相关工作

2.1 真实网页任务与评测环境

网页 Agent 的研究首先需要可复现的任务环境。Mind2Web [1] 收集跨 137 个网站的真实网页操作轨迹，强调真实 HTML 往往很长，模型需要先筛选候选元素再预测动作。WebArena [2] 构建了可自托管的真实感网页环境，通过端到端任务成功率评价 Agent。VisualWebArena [3] 在此基础上加入视觉密集任务，说明仅依赖 DOM 文本难以理解图标、图片、空间布局和视觉状态。

WebVoyager [4] 更接近本项目的目标：它直接面向真实在线网站，使用网页截图和 Accessibility Tree 构建观察，并用大模型执行连续网页操作。BrowserGym/AgentLab [5] 则提供标准化 WebAgent 实验接口，对 observation space、action space、任务初始化和批量实验管理有很强参考价值。我们的系统没有直接复用这些 benchmark，而是吸收其任务组织方式：每个任务显式定义起始 URL、自然语言目标、最大步数、成功标准、禁止动作和 trace 输出。

2.2 界面理解与 Grounding

网页操作的关键瓶颈是 grounding：模型可能知道“应该点击搜索框”或“应该展开某个学校”，但无法稳定定位到页面上的准确元素。SeeAct [6] 研究了多模态模型作为网页 Agent 时的 grounding 问题，提出将视觉理解与候选元素定位结合。Set-of-Marks [7] 通过在图像上叠加编号标记，把“点击某个视觉区域”转化为“选择某个编号”，降低了直接坐标预测的难度。

Web-VLMAgent 采用类似思想：DOM 提取器从页面中筛选按钮、链接、输入框、文本框、下拉框以及常见 ARIA role 元素，记录每个候选元素的编号、标签、角色、文本、选择器、bbox、可见性和可用性；SoM 模块再把可见且面积合理的元素绘制成编号框。模型最终只能通过 `target_id` 引用候选元素，执行器会在真实点击前再次确认该编号存在、可见且可用。

2.3 推理-行动闭环

ReAct [8] 将推理和行动交替组织，使模型在每一步先根据观察产生简短 thought，再选择一个工具动作。这种方式适合网页任务，因为网页状态会在每次点击、输入、滚动后改变，模型不能一次性可靠预测完整动作序列。我们采用“单步 ReAct + 外部循环”的方式：模型每次只输出一个 JSON 动作，循环逻辑、浏览器执行、失败检测和 trace 保存由宿主程序控制。

与端到端坐标 Agent 相比，这种设计更工程化，也更容易调试。若模型输出非法 JSON，Agent 层会保存 `raw_response.txt`，自动重试一次；若仍失败，则退化为安全 `wait` 动作。若模型输出了不存在的 `target_id`，执行器会拒绝执行并记录 grounding error，而不是盲目点击。

2.4 GUI Agent 的扩展启发

OSWorld [9]、Windows Agent Arena [10] 和 AndroidWorld [11] 把 Agent 从网页扩展到桌面和移动系统，强调真实环境、任务初始化、状态检查和轨迹级错误分析。UI-TARS [12]、OmniParser [13]、SeeClick [14] 等工作进一步说明，界面元素解析和视觉 grounding 是 GUI Agent 的核心能力。虽然本项目聚焦浏览器网页，不引入 OSWorld 或 PyAutoGUI，但借鉴了这些工作的两个原则：一是操作必须在真实环境中执行验证，二是失败必须能追溯到具体观察和动作。

表 3 代表性相关工作与本项目对应模块

方向	代表工作	对本项目的启发
网页任务数据集	Mind2Web [1]、WebLINX [15]	真实网站任务、候选元素筛选、动作历史建模
网页评测环境	WebArena、VisualWebArena	端到端成功率、视觉网页任务、执行式验证
真实网页 Agent	WebVoyager、SeeAct	截图 + A11y + VLM 的操作闭环，元素 grounding
视觉标注增强	Set-of-Marks	将视觉区域转化为编号选择，降低坐标定位难度
推理行动框架	ReAct	单步观察、思考、行动、反馈的多步任务循环
实验框架	BrowserGym/AgentLab	标准化 observation/action space、批量实验和报告输出
通用 GUI Agent	OSWorld、UI-TARS、OmniParser	轨迹级错误分析、屏幕解析、状态检查

2.5 本项目与已有工作的区别

本项目不是构造一个新的大型 benchmark，也不训练新的 VLM；目标是在课程项目规模内实现一个可运行、可评测、可展示的网页多模态 Agent 原型。因此我们更强调工程闭环和可复盘性：真实网页任务来自 CSRankings、Bilibili 和旅行网站；模型通过 OpenAI-compatible API、DeepSeek 或 Gemini 等代理接入；浏览器执行由 Playwright 负责；评测脚本把确定性任务和开放任务分开处理。这样的设计使系统既能满足课程项目的要求，也能作为后续扩展到更多模型和更多网站的实验平台。

3 系统设计与实现

3.1 总体架构

Web-VLMAgent 的代码按浏览器执行、感知标注、模型调用、Agent 决策、评测与可视化回放分层组织。核心运行入口覆盖单任务、批量评测和多模型套件。单任务运行时，系统从 YAML 任务或命令行读取起始 URL、任务指令、最大步数、模型 provider 和 observation mode；随后启动浏览器，进入 Observe–Plan–Act–Check–Verify 循环。

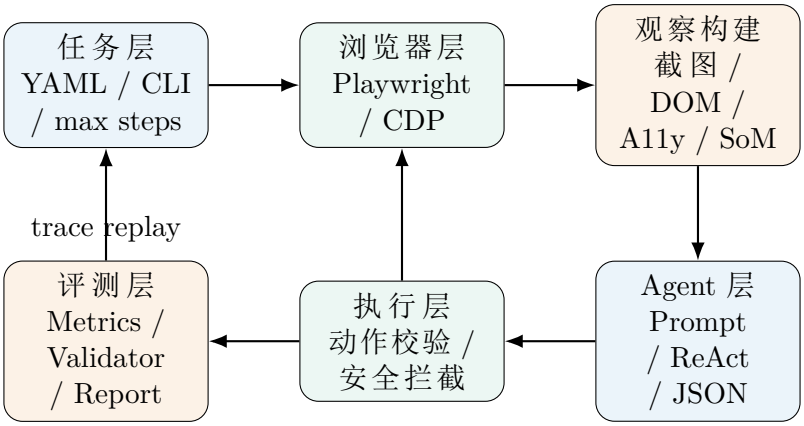


图 2 Web-VLMAgent 模块架构

表 4 核心目录与职责

目录/文件	职责
`browser/`	Playwright/CDP 浏览器环境、DOM 候选元素提取、动作 schema 与执行校验
`perception/`	Set-of-Marks 标注图生成、候选元素过滤
`models/`	OpenAI-compatible、DeepSeek、Gemini 客户端，统一 chat completion 接口
`agents/`	ReAct prompt、单步 Agent、Progress Checker、Final Verifier、任务 checklist、最终 HTML 报告生成
`eval/`	单任务 runner、批量 runner、指标统计、validator、summary/report 生成
`scripts/`	demo、SoM demo、单步 Agent、多步任务、批量评测和模型 suite 命令行入口
`tasks/`	8 项核心任务的 YAML，以及单独存放的 1 项补充探索任务
`webui/`	Trace Viewer / Workbench，可查看运行状态与轨迹

3.2 观察空间

观察空间决定模型“看见”什么。真实网页任务中，单一模态很容易失效：截图能表示视觉布局，却缺少可执行选择器；DOM 有元素结构，却不一定反映视觉遮挡和用户实际看到的内容；Accessibility Tree 有语义标签，但不总是完整；页面文本适合阅读任务，却难以定位点击对象。因此系统采用混合观察。

表 5 观察空间组成

观察项	保存文件/字段	用途
原始截图	`screenshot.png`	保留页面真实视觉状态、图片、布局、遮挡和弹窗
SoM 标注图	`marked_screenshot.png`	在可见候选元素上绘制编号框，帮助视觉模型选择 `target_id`
候选元素列表	`candidates.json`	提供 `element_id`、tag、role、text、selector、bbox、visible、enabled
Accessibility Tree	A11y tree	提供角色、标签、标题、对话框等语义结构
页面文本片段	`page_text.txt` / prompt excerpt	用于阅读、证据抽取、跨页面记忆
动作历史	prompt 中的 compact history	记录最近动作、URL、执行结果、进展信号和证据摘要
任务 checklist	`checklist.json`	将开放任务拆成可跟踪的证据项，减少长任务遗忘

系统支持三种 observation mode：`text` 只发送候选元素、页面文本和 A11y 信息，适合无视觉能力的语言模型；`vision` 额外把原始截图和 SoM 图作为图像输入，适合 Gemini、GPT、Claude、Qwen-VL 等视觉模型；`auto` 根据 provider 自动选择。这样可以在同一任务上比较纯文本模型与视觉语言模型的差异。

3.3 候选元素与 Set-of-Marks

DOM 提取器至少覆盖 button、a、input、textarea、select，以及 role 为 button、link、textbox 的元素和可编辑元素。每个候选元素被赋予稳定的 `element_id`，并保存页面 bbox。SoM 绘制阶段会过滤不可见元素、面积过小元素和屏幕外元素，然后用 Pillow 绘制轻量编号框。这样模型不需要输出坐标，只需选择编号；执行器再根据 selector 滚动到元素并点击/输入。

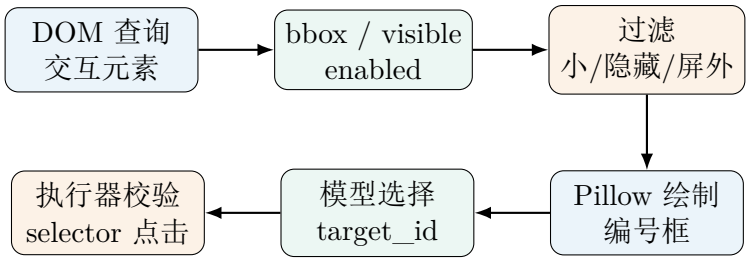


图 3 候选元素与 Set-of-Marks grounding 流程

3.4 动作空间与安全校验

动作空间采用结构化 JSON，模型只能输出下列动作：`click`、`type`、`scroll`、`press`、`back`、`wait`、`answer`、`done`、`ask_user`、`navigate`、`search`。其中 `click` 和 `type` 必须携带合法 `target_id`；其他动作强制将 `target_id` 置空。

表 6 动作空间与执行约束

动作	关键字段	执行约束
`click`	`target_id`	目标必须存在、可见、可用；禁止点击登录、支付、购买、删除、提交等高风险控件
`type`	`target_id`, `text`, `key`	文本不能为空；禁止向密码、信用卡、地址、真实联系方式等敏感输入框填入信息
`scroll`	`direction`	仅允许 up/down；用于寻找更多候选或页面文本
`press`	`key`	只执行键盘按键，例如 Enter 或 Escape
`navigate`	`text`	必须是完整 http/https/file URL，避免模型拼接危险或不完整地址
`search`	`text`	通过搜索引擎查询公开信息，不替代指定网站步骤
`answer/done`	`text`	触发 Final Verifier；答案必须尽量由当前页面或历史证据支持
`ask_user`	`text`	遇到验证码、登录墙、模糊选择或安全风险时暂停询问

动作校验由 `browser/actions.py` 中的 Pydantic schema 和规则函数完成。高风险点击词包括 delete、remove、purchase、buy、pay、checkout、order、subscribe、login、sign in、submit 等；敏感输入词包括 password、credit card、cvv、address、ssn 等。该设计保证即使模型产生危险计划，执行器也不会直接执行。

3.5 ReAct Prompt 与单步决策

Agent prompt 明确要求模型“只返回一个 JSON 对象，不要 markdown，不要代码”。输入给模型的信息包括任务指令、当前 URL、截图路径或图像输入、候选元素列表、页面文本、Accessibility Tree、人类提示、任务 checklist 和历史动作摘要。系统 prompt 还包含恢复策略：若页面被弹窗、cookie banner、登录浮层或验证码遮挡，优先选择安全关闭、Escape、等待或 `ask_user`；若最近动作没有推进页面状态，不要重复同一动作。

Algorithm 1: 单任务多步运行流程

Input: 起始 URL u_0，任务指令 I，最大步数 T，模型客户端 M
Output: run summary, trace artifacts, final answer

```
打开浏览器并访问 u_0
初始化历史动作、任务 checklist 和 trace 目录
for t = 0 to T − 1 do
    截图，提取候选元素、页面文本和 Accessibility Tree
    绘制 SoM 标注图并保存当前 step artifacts
    构造 ReAct prompt，调用模型 M 生成一个 JSON 动作
    解析动作；若非法 JSON，则保存原始响应并重试一次
    校验动作 schema、安全规则和 target_id
    执行动作并保存 execution_result.json
    调用 Progress Checker 判断是否推进页面状态
    if 动作为 answer 或 done then
        调用 Final Verifier 检查答案证据
        if 验证通过 then
            写入成功 summary 并结束
        end
    end
    if 连续卡住或达到恢复条件 then
        根据 checklist 生成提示、继续、询问用户或失败退出
    end
end
若超出最大步数，生成部分结果并保存失败类型
```

3.6 Progress Checker 与 Final Verifier

Progress Checker 通过 URL、候选元素数量、候选文本签名、截图 hash 等信号判断动作是否改变页面；同时检测重复动作、搜索/导航循环、URL ping-pong、导航参数被网站拒绝等情况。当动作连续没有改变有用状态时，系统会把恢复提示加入下一步 prompt，或在可配置情况下请求用户帮助。

Final Verifier 用于避免模型“自信地宣布完成”。当模型输出 `answer` 时，Verifier 会检查答案文本、URL、页面文本和历史证据是否匹配任务关键词；对中文编码可能受 API 代理影响的场景，还实现了 ASCII/数字片段回退匹配。当模型输出 `done` 时，Verifier 会检查近期是否有成功执行动作、任务中的字面量是否出现在证据中。CSRankings 等确定性任务还会在运行结束后由 `eval/validators.py` 再做答案级校验。

3.7 Trace 格式与可视化

每个 run 保存到 `traces/task_run/{run_id}/` 或用户指定目录。每一步的目录结构如下：

表 7 单步 trace 文件

文件	含义
`screenshot.png`	原始页面截图
`marked_screenshot.png`	带候选元素编号的 SoM 标注图
`candidates.json`	当前可交互候选元素列表
`accessibility_tree.txt`	当前页面可访问性树
`page_text.txt`	渲染文本与候选文本片段
`prompt.txt`	发送给模型的文本 prompt
`raw_response.txt`	模型原始输出，保留非法 JSON 与重试信息
`parsed_action.json`	解析后的结构化动作
`execution_result.json`	执行是否成功、执行前后 URL、错误类型和异常信息
`progress_check.json`	页面是否推进、是否重复/卡住、进展信号
`final_verification.json`	终止动作的证据检查结果

批量评测完成后，`eval/report_generator.py` 会生成 `summary.csv`、`summary.md` 和 `trace_report.html`。HTML 报告把每个 run 展开为 step-by-step 轨迹，包含截图、标注图、动作、执行结果、进展检查和最终验证，满足课程要求的“可视化操作轨迹（截图序列 + 动作序列）”。

4 任务设计与评测方案

4.1 任务库设计原则

课程要求每个网站至少 3 个任务。我们没有只选择简单搜索任务，而是按 easy、medium、hard 三档构建任务库，使任务难度逐步增加：easy 任务侧重页面定位和单点信息抽取等简单任务；medium 任务往往需要跨页面查找、比较和结构化字段补全；hard 任务强调长链推理、跨来源验证、候选集合筛选和最终报告生成。

核心评测包含 CSRankings 两项、Bilibili 三项与广州旅游三项任务，合计八项，分别存于三个主线 YAML 文件。另将 CSRankings 跨方向统计单独存于探索任务目录；由于题目与判分设置需要进一步审查，该任务不进入核心完成率。每项任务均定义任务标识、网站、难度、起始地址、自然语言说明、最大步数、成功标准和禁止动作。

表 8 8 项核心评测任务与 1 项补充探索任务

场景	难度	task id 摘要	任务摘要
CSRankings	easy	NLP professor	在 2020–2025 年美国 NLP 排名最高学校中寻找论文数最多、排名最靠前的教授，确定性答案为 Graham Neubig。
CSRankings	medium	CN ML scholar	找 2015–2025 年中国大陆高校 Machine Learning 排名最高学校中排名最高学者，并补全职位、邮箱、引用量、H-index 和最高引论文标题。
CSRankings	探索	AI/Sys. faculty	不计入核心指标；比较美国高校在 8 个 AI/Systems 方向前 30 名中的均衡性，计算中位数、平均值和最差排名，再找覆盖方向最广的 faculty。
Bilibili	easy	public video	从 B 站主页进入公开视频页，返回标题、链接和可见简介；若无简介需明确说明。
Bilibili	medium	AI top 3	打开“人工智能”搜索结果页，读取综合排序前三个 UP 主视频的可见信息，并分析受众与理由。
Bilibili	hard	bangumi report	浏览至少 5 个热门原创番剧详情，选 3 个外部站核验，最终选择 1 个推荐对象，获取相关视频下载链接并生成 HTML 展示页。
广州旅游	easy	top attraction	在携程查找广州景点页，返回排序第 1 景点的名称、评分、地址或区域、开放时间或门票提示和详情页 URL。
广州旅游	medium	Chimelong	只使用携程景点页，至少打开 3 个广州长隆相关景点详情，比较评分、地址、开放时间、票务提示并推荐 1 个。
广州旅游	hard	two-day plan	从广州南站进出，基于真实网页规划两天一晚行程；查酒店、餐厅、官方景点页面，给出交通、预算、雨天替换和来源 URL。

4.2 难度划分

难度不单由步数决定，而由“状态空间、证据要求、判分复杂度”共同决定。CSRankings 核心任务侧重教授定位和跨来源档案补全；补充探索任务涉及 8 个方向的学校排名、统计量计算与 faculty 覆盖统计，当前用于检查题目与判分的适用性。Bilibili 与旅游任务更开放，hard 任务不仅需要浏览多个候选页面，还要跨来源核验和生成最终展示材料。

表 9 Easy/Medium/Hard 分层标准

难度	典型能力	评测重点
Easy	目标明确，少量页面跳转，单点信息抽取	元素定位、滚动、基础阅读、答案是否精确。
Medium	多页面跳转、字段补全、候选比较	跨页面记忆、来源记录、结构化输出完整性。
Hard	长链任务、跨来源验证、复杂计算或报告生成	计划分解、证据覆盖、避免循环、开放结果可复核性。

4.3 评测指标与判分口径

批量运行后，`eval/metrics.py` 从每个 `run_summary.json` 和 step trace 构造 run record，并计算以下指标：

- **Task Success Rate (SR)**：harness 判定成功的 run 数 / 总 run 数，反映主循环是否顺利停止并生成答案；
- **Validated Success Rate**：对确定性任务按答案规则复核后的成功率；
- **Average Steps / Average Time**：平均动作步数与耗时；
- **Action JSON Validity Rate**：解析成功且未使用 fallback 的动作步数占比；
- **Grounding Error Rate**：因 `target_id` 不存在、不可见或不可用导致的定位错误占比；
- **Correction/Retry Success Rate**：触发重试或恢复机制后最终成功的比例；
- **Failure Type Distribution**：将失败归类为 perception、planning、execution、verification、website 五类。

CSRankings 的两项核心任务属于封闭答案任务，可由脚本严格复核；补充探索任务单独记录，不参与核心指标。Bilibili 与广州旅游规划任务具有开放性，页面内容会随时间、推荐系统、地区和登录状态变化，因此先自动检查字段覆盖和来源 URL，再结合 trace 进行人工或 LLM-as-judge 复核。本文明确区分 **harness success**（流程正常结束）和 **validator success**（答案通过严格规则）；二者不能混用。

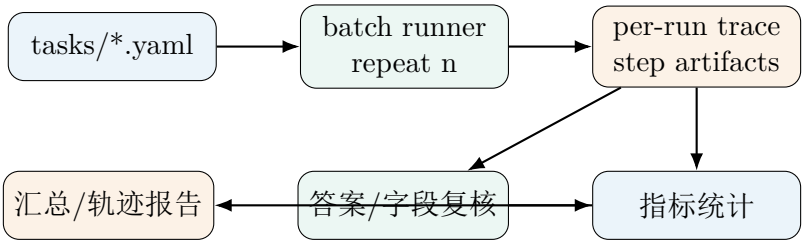


图 4 批量评测与报告生成流程

4.4 第一轮评测：DeepSeek V4 Pro 八项核心任务重复评测

DeepSeek V4 Pro 对八项核心任务各运行 10 次，共 80 次，记录到 76 次 harness 完成，完成率为 **95.00%**。CSRankings 两项核心任务共 20 次均完成；Bilibili 三项任务共 30 次均完成；广州旅游三项任务共 30 次完成 26 次。四次未完成分别为广州 medium 的三次规划失败和广州 hard 的一次感知失败。

原始实验还对补充探索任务运行了 10 次。由于该题的任务定义与判分设置需要进一步核验，这部分记录与八项核心任务分开，不用于推断模型成功率。原九项混合批次的平均时间、平均步数及失败比例也不直接沿用为核心任务指标。

表 10 DeepSeek V4 Pro 正式评测场景级结果

场景	运行次数	成功次数	成功率	说明
CSRankings	20	20	100.00%	两项核心任务各运行 10 次，均完成。
广州旅游规划	30	26	86.67%	medium 失败 3 次，hard 失败 1 次。
Bilibili	30	30	100.00%	三个难度均成功。
合计	80	76	95.00%	核心任务未完成 4 次。

表 11 DeepSeek V4 Pro 正式评测任务级结果

场景/难度	任务	次数	成功	成功率	平均步数	备注
CS easy	NLP top professor	10	10	100%	3.4	平均用时 186.19 秒。
CS medium	CN ML scholar profile	10	10	100%	8.1	平均用时 308.29 秒。
GZ easy	Ctrip top attraction	10	10	100%	5.1	平均用时 235.94 秒。
GZ medium	Chimelong sights comparison	10	7	70%	10.7	规划失败 3 次。
GZ hard	two-day real web plan	10	9	90%	34.1	感知失败 1 次,平均用时 1119.67 秒。
Bili easy	public video intro	10	10	100%	2.2	平均用时 235.37 秒。
Bili medium	AI search audience	10	10	100%	2.6	平均用时 287.21 秒。
Bili hard	bangumi HTML recommendation	10	10	100%	26.25	平均用时 708.60 秒。

核心任务结果表明，CSRankings 两项信息检索、广州 easy 与 Bilibili 三项任务的

完成过程较稳定；广州 medium/hard 仍暴露跨页面导航、信息覆盖与证据管理的问题。长任务还具有更高的操作成本，例如广州 hard 平均 34.1 步、1119.67 秒，Bilibili hard 平均 26.25 步、708.60 秒。完成率应与任务范围、步数、用时和校验方式共同解读。

4.5 GPT-5.5 八项核心任务视觉基线

GPT-5.5 以 vision observation mode 对八项核心任务各运行一次，均记录为完成，平均步数为 9.75。两项 CSRankings 核心任务通过确定性答案校验；六项开放任务的字段覆盖均为 1.0，内容真实性仍需结合 trace 复核。该基线每题仅一次，不能等同于重复运行可靠性或严格答案正确率。原始混合批次还包含一项探索任务的单次运行；该运行不进入表中核心指标。由于现有汇总没有提供完整的逐任务用时，不为核心子集复用原混合批次的平均用时。

表 12 GPT-5.5 八项核心任务基线结果

任务	难度	success	步数	复核结论
CSRankings NLP professor	easy	成功	3	validator 通过，答案为 Graham Neubig。
CSRankings ML profile	medium	成功	15	validator 通过，朱军档案字段匹配。
Bilibili public video intro	easy	成功	2	字段覆盖 1.0，需结合 trace 复核具体视频页。
Bilibili AI search top 3 audience	medium	成功	1	字段覆盖 1.0，从搜索页可见信息完成分析。
Bilibili bangumi HTML	hard	成功	20	字段覆盖 1.0，完成候选浏览、外部核验和 HTML 输出。
Ctrip top attraction	easy	成功	4	字段覆盖 1.0，返回景点、评分、地址与详情页。
Ctrip Chimelong compare	medium	成功	6	字段覆盖 1.0，完成多景点详情页对比。
Guangzhou two-day plan	hard	成功	27	字段覆盖 1.0，完成酒店、餐饮、景点与路线规划。

4.6 十模型横向评测

我们随后选取 10 个多样化模型进行横向评测，覆盖 OpenAI、Google Gemini、Anthropic Claude、Qwen、DeepSeek、Kimi、GLM 和豆包系列。视觉模型为 GPT-4o mini、GPT-5 mini、Gemini 2.5 Flash、Claude Haiku 4.5、Qwen3-VL-32B 与豆包 Vision Pro；文本模型为 DeepSeek-V4-Pro、Qwen3.7-Max、Kimi-K2.6 与 GLM-5.2。完整 API 名称见表 13。前 6 个按 vision mode 运行，后 4 个按 text mode 运行。历史批次同时包含八项核心任务和一项补充探索任务，与 10 个模型构成 90 个计划组合。核心评测的设计范围是 80 个模型–任务组合，探索部分另有 10 个；计划规模不等于有效样本数。

我们在此次评测中也遇到了诸如中转站超时、内容安全拦截、上下文超限等服务层问题，为了避免误计为模型能力失败，我们把每个组合标记为 **valid**、**invalid** 或 **incomplete**。表 13 保留历史混合批次的汇总，用于追溯原始实验覆盖：CS 列同时包含两项核心任务和一项探索任务，开放列包含六项核心任务，数对按“有效样本数/通过或完成数”排列。汇总合计为 66 个有效、22 个无效和 2 个未完成。该表未逐项标明部分模型的探索任务状态，不能据此精确重建八任务核心子集，也不能将 CS 列直接用作核心任务排行榜。

表 13 十模型历史批次记录：核心任务与补充探索任务的混合汇总

模型	模式	CS 严格成功	开放完成	无效	未完成	有效样本中的主要观察
gpt-4o-mini	vision	3/0	6/2	0	0	探索任务和部分开放长任务出现动作循环。
gpt-5-mini	vision	3/2	6/4	0	0	核心 easy/medium 较稳；探索任务与广州 hard 需进一步检查。
gemini-2.5-flash	vision	3/1	6/3	0	0	可完成部分阅读任务，但在广州 medium/hard 与 B 站 hard 中出现证据收集循环。
claude-haiku-4-5	vision	3/1	0/0	6	0	六项开放任务输入超限；CS 记录含探索任务，不能直接按核心范围比较。
qwen3-vl-32b	vision	3/2	6/4	0	0	CS easy/medium 与 B 站任务较好，广州 medium/hard 的真实网页导航仍不稳定。
doubao-vision-pro	vision	0/0	0/0	9	0	图文总 token 超限，当前配置下无可比较有效样本。

模型	模式	CS 严格成功	开放完成	无效	未完成	有效样本中的主要观察
DeepSeek-V4-Pro	text	3/2	6/5	0	0	纯文本模式表现稳健，说明 DOM/A11y/page text 对信息型任务有较强支撑。
qwen3.7-max	text	2/2	5/5	2	0	有效样本均完成，但广州 hard 出现内容安全拦截，需要补跑。
kimi-k2.6	text	2/2	3/2	2	2	有效样本尚可，但服务超时和长任务未完成影响正式统计。
GLM-5.2	text	2/2	4/3	3	0	easy 与 B 站 medium 较好；广州 medium 超时、广州 hard 被内容拦截。

历史实验提供以下定性观察。第一，流程完成与答案正确需要分别记录；确定性核心任务采用规则校验，开放任务还需结合字段与来源证据复核，补充探索任务不用于核心成功率推断。第二，文本观察中的 DOM、A11y 和页面文本可以支持信息型任务，但当前视觉与文本模式的输入预算不同，不能据此推断视觉输入的因果效果。第三，开放长任务中的重复导航、过早回答和来源遗漏表明，规划与证据管理仍需改进。上述观察用于系统诊断，不构成八项核心任务上的完整模型排名。

4.7 十模型评测中的无效样本与风险

在报告有效样本结论之后，还必须单独说明不可归因为模型能力的技术问题。第一类是 **API read timeout**，如 `kimi-k2.6` 的 Bilibili medium、`GLM-5.2` 的广州 medium：请求已发出，但中转站或上游模型在超时时间内没有返回。第二类是 **HTTP 400 内容安全拦截**，如 `qwen3.7-max` 和 `GLM-5.2` 在广州 hard 中被上游安全策略拦截；真实网页文本中下载链接、支付、登录、预约等敏感词可能共同触发代理侧策略，不能直接解释为规划能力差。第三类是 **上下文或图文 token 超限：**`claude-haiku-4-5` 的 6 个开放任务、`doubao-1-5-vision-pro-32k` 的多个任务因此无效。

造成 token 超限的直接原因是当前 vision mode 的 observation 较重：每一步会同时发送任务、动作历史、候选元素、页面文本、Accessibility Tree、raw screenshot 与 marked screenshot。以 Bilibili easy 为例，Claude 的第一步 prompt 文本约 55 KB、Accessibility Tree 约 36 KB、候选元素约 25 KB，两张截图接近 1.9 MB；经 base64 和代理侧估算后约为 428k tokens，超过上下文限制。我们当前的 observation space 内容非常丰富，一方面这可以为强上下文模型提供更加丰富的信息，但是另一方面这也导致了短上下文模型输入超限而造成任务失败。后续如果想评测此类模型，可能需要专门针对其简化 observation space 设计。

5 错误分析、局限与展望

5.1 从有效样本看失败模式

八项核心任务的重复评测显示，基础检索与部分内容分析任务较稳定，跨页面比较与规划仍需要改进。补充探索任务涉及跨方向统计与复杂判分；由于其题目设置需要进一步核验，该任务仅用于检查任务定义、证据组织与执行轨迹，不据此形成模型能力排名。核心评测与探索记录应分别报告。

开放任务中，Bilibili 和广州旅游的 medium/hard 更强调跨页证据管理。DeepSeek V4 Pro 在 Bilibili 三个任务上均计为 10/10 成功，在广州 hard 上 9/10 成功，但广州 hard 平均步数达到 34.10，说明开放长链任务即使能完成，也会显著放大证据组织成本。十模型实验中的典型失败包括在搜索页与详情页间重复跳转、只读到部分候选项就提前回答、或收集了信息却没有记录其来源 URL。这说明 JSON 动作合法并不保证任务真的完成，进度检查、任务清单和 trace 证据共同构成了评测闭环。

5.2 失败类型与改进方向

表 14 失败类型、典型表现与改进方法

类型	典型表现	可能改进
感知失败	看不到弹窗、误读截图、候选元素缺失或编号遮挡	改进元素过滤、A11y 补充、截图裁剪、多尺度 SoM；对图像密集页面加入 OCR。
定位失败	选择不存在或不可见的 `target_id`，点击后无响应	更严格候选生成，失败后自动重新观察，将定位错误反馈给模型。
规划失败	重复搜索、遗漏任务字段、过早回答、长链任务忘记证据	引入 checklist、任务分解、结构化 notebook，将 planner 与 executor 分离。
执行失败	网站跳转异常、按钮被遮挡、网络超时、页面加载慢	更稳定的等待策略、弹窗处理、重试机制和环境错误分类。
验证失败	harness 成功但答案错误，开放任务字段覆盖不等于真实完成	确定性 validator、LLM-as-judge、人工复核和 trace 证据约束。
网站/服务失败	站点访问异常、API timeout、HTTP 400、安全策略或上下文上限	与模型能力错误分离记录，保留原始报错并在环境恢复后补跑。

八项核心任务的 DeepSeek V4 Pro 重复评测共 80 次，完成 76 次，未完成 4 次，分别为三次规划失败和一次感知失败；补充探索任务的记录不计入该比例。十模型有效样本也没有出现大量结构化动作解析失败。因此，瓶颈主要转移到长链任务的计划分解、证据组织和最终严格校验。另一方面，API timeout、内容拦截和图文 token 超限说明代理服务、模型接口和 observation 预算同样会决定评测是否有效。因此，报告结果时应同时呈现有效样本结论、无效样本数量及其原因，而不能把所有失败压缩为单一成功率。

5.3 当前局限

当前系统已经具备课程项目所需的完整闭环，但仍有明显局限。

1. **开放任务自动评测仍偏弱。**字段覆盖和 URL 数量检查只能判断答案形式，无法完全判断证据真实性和推荐合理性；Bilibili hard 与旅游 hard 仍需人工或更强 LLM judge 结合 trace 复核。
2. **跨模型输入尚未完全归一。**视觉模型需要处理截图与 SoM，文本模型只接收 DOM、A11y 和页面文本；这符合各自接口能力，但 token 预算与输入信息量仍会影响横向可比性。
3. **真实网站不可控。** B 站推荐、携程页面和搜索结果随时间、地区、登录状态和反爬策略变化，导致重复运行的可比性下降。
4. **复杂页面操作仍不稳定。**筛选器、下拉框、悬浮弹窗和懒加载列表会使候选元素提取不完整；模型有时会依赖猜测 URL 参数，而不是逐步操作可见控件。
5. **复杂报告生成可能放大错误。**当最终答案含有未核实事实时，HTML report 会使结果显得更完整，因此必须在生成前保留严格证据约束。

5.4 后续工作

第一，评测更严格。对 CSRankings 类封闭任务继续维护确定性 validator；对 Bilibili 和旅游规划任务，引入同时读取最终答案、trace 中访问 URL、截图序列和任务标准的 LLM-as-judge，并报告自动层与复核层两套结果。

第二，感知更稳定、更轻量。优化候选元素过滤和 SoM 布局，区分 viewport 内、屏幕外可滚动及弹窗元素；对视觉模型采用压缩单图和高价值元素摘要，对纯文本模型清洗 A11y 与页面文本，从而降低上下文超限风险。

第三，规划更可控。将复杂任务拆为 planner 和 executor 两层：planner 先生成 checklist 与证据需求，executor 逐项收集证据；每完成一个字段就写入带 URL 和截图引用的结构化 notebook，最终答案仅引用已核验事实。

5.5 总结

Web-VLMAgent 已从最小浏览器闭环扩展为可运行、可评测、可回放的网页多模态 Agent 系统。它将网页截图、DOM 候选元素、Accessibility Tree、SoM 标注图和页面文本组合成混合观察空间，用结构化 JSON 动作和安全校验约束模型行为，并通过 Progress Checker、Final Verifier、YAML 任务管理、批量评测和 HTML trace report 支撑课程要求的成功率统计、失败分类和可视化轨迹。

从实验结果来看，Web-VLMAgent 已能够支撑真实网页任务中的系统化测试。八项核心任务的 DeepSeek V4 Pro 重复评测取得 76/80 的 harness 完成结果；GPT-5.5 在同一核心范围的单次基线中完成 8/8，平均 9.75 步，但开放任务仍需语义与来源复核。十模型历史实验提供了输入预算、规划稳定性和服务异常方面的诊断信息；其混合汇总

不应被当作八任务核心排行榜。另保留一项探索性任务用于检查复杂题目的定义与判分。真实网页 Agent 的评估需要同时交代任务范围、中间轨迹、证据来源及环境因素，才能准确说明所观察到的能力与局限。

参考文献

[1] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems 36*, 2023.

[2] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[3] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905. Association for Computational Linguistics, 2024.

[4] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890. Association for Computational Linguistics, 2024.

[5] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The BrowserGym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2024.

[6] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61349–61385. PMLR, 2024.

[7] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023.

[8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[9] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

[10] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Jang, and Zack Hui. Windows Agent Arena: Evaluating multi-modal OS agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.

[11] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.

[12] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

[13] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for pure vision based GUI agent. *arXiv preprint arXiv:2408.00203*, 2024.

[14] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332. Association for Computational Linguistics, 2024.

[15] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. WebLINX: Real-world website navigation with multi-turn dialogue. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 33007–33056. PMLR, 2024.